

EMPIRICAL TURING COMPLETENESS IN A PRODUCTION TRANSFORMER VIA IN-CONTEXT LEARNING

PREPRINT, COMPILED MAY 11, 2026

Christian T. J. Lewis^{1*}

¹Evolving Programs

Abstract

Pérez et al. (2021) proved that a specially constructed transformer with positional encodings is Turing complete. We show that this property is not merely theoretical: Anthropic’s Claude 3 Opus, a production large language model, exhibits universal computation empirically through in-context learning alone. Given twelve example tapes covering ten of the 256 elementary cellular automata rules, Opus learns the generalized update algorithm and executes it correctly for all 256 rules on arbitrary inputs — including Rule 110, proven Turing complete by Cook (2004), and Rule 0, whose degenerate output rules out distributional pattern matching as an explanation. In a separate experiment, the model learns binary multiplication from structured examples and executes it correctly on random operands up to 27 bits, producing correct 54-bit products. No chain-of-thought reasoning or scratchpad is used in either case; computation proceeds entirely within the autoregressive output stream. The results bridge the gap between theoretical Turing completeness constructions for transformers and the empirical capabilities of trained models, demonstrating that gradient descent on natural language produces systems that can function as universal computers given sufficient context.

Keywords transformer, Turing completeness, in-context learning, cellular automata, attention

1 INTRODUCTION

A central question in the theory of large language models is whether the transformer architecture is capable of general-purpose computation, or whether its apparent reasoning abilities are artifacts of sophisticated pattern matching over training data. Multi-digit multiplication has become a canonical test case for this question: Dziri et al. (2024) argued at NeurIPS that transformer performance on compositional tasks like multiplication “rapidly decays with increased task complexity,” reporting that GPT-4 achieves only 59% accuracy on 3-digit by 3-digit multiplication. This has been widely interpreted as evidence of a fundamental architectural limitation.

Pérez, Barceló, and Marinkovic (2021) answered the theoretical version of this question affirmatively, proving that transformer networks with positional encodings are Turing complete. Their construction is explicit: given any Turing machine, one can build a transformer that simulates it. However, theoretical constructions and the behaviors of trained models are fundamentally different objects. A proof that a system *can* be configured for universal computation does not establish that gradient descent *will* produce such a configuration, or that the resulting model can be prompted to exhibit it.

Subsequent theoretical work has tightened the picture. Li and Wang (2025) showed that even constant bit-size transformers are Turing complete given sufficiently long context windows and chain-of-thought steps. Merrill (2023) characterized saturated transformers as constant-depth threshold circuits, establishing both capabilities and limi-

tations of the attention mechanism under different precision regimes. Yet the empirical question remained open: does a production model, trained on natural language, actually contain the computational machinery necessary for universal computation?

We demonstrate that it does. In April 2024, we showed that Anthropic’s Claude 3 Opus can simulate all 256 elementary cellular automata rules — including rules known to be Turing complete — by learning the generalized update algorithm from twelve in-context examples. In May 2024, we extended this to binary multiplication, showing the model can learn and execute arithmetic algorithms on operands up to 27 bits. In both cases, the model receives no English-language explanation of the algorithm, no chain-of-thought scaffolding, and no external tools. It induces each algorithm from patterned examples alone and executes it via autoregressive generation, with the output stream serving as the computational tape.

2 METHOD

2.1 General Approach

Both experiments follow the same methodology. The model receives a structured prompt consisting of complete worked examples — input-output traces of an algorithm applied to specific inputs — and is then asked to execute the same algorithm on a new, unseen input. The examples are formatted as structured program traces with dense positional indexing. No natural-language description of the algorithm is provided; the model must induce it from the pattern of the examples alone.

This dense indexing is not arbitrary; it is a necessary response to the attention saturation problem identified by Merrill (2023). Without explicit positional anchors, attention distributes across long sequences and the model misreads its own previously emitted state, introducing copy errors that propagate through subsequent steps. Every redundant label in the tape design gives attention heads a stronger target to lock onto, reducing leakage onto adjacent indices.

All experiments use Claude 3 Opus (claude-3-opus-20240229) at temperature 0, with approximately 30,000 input tokens and a 4,096-token output budget per API call. When a computation exceeds this budget, the response is chained: the model’s output is appended to the input context and the model is prompted to continue. This works because the computation is Markovian — each step depends only on the current tape state, so the model can resume from any cut point without loss of information. The full 200,000-token context window is available as effective tape across chained calls. Outputs are evaluated by exact match against ground truth computed by a conventional program: any single incorrect token causes the entire test to fail.

2.2 Elementary Cellular Automata

An elementary cellular automaton (ECA) operates on a one-dimensional binary tape. At each time step, every cell updates according to a fixed rule that maps each possible three-cell neighborhood (left neighbor, cell, right neighbor) to a binary output. Since there are 8 possible three-bit neighborhoods, there are $2^8 = 256$ possible rules, conventionally indexed by the decimal value of their 8-bit output string (Wolfram, 2002). Rule 110 has been proven Turing complete (Cook, 2004), meaning it can simulate any Turing machine given a suitable initial configuration and sufficient tape.

The model receives twelve complete worked examples, each demonstrating one ECA rule applied to a 12-cell tape for 12 steps. The examples cover ten distinct rules. Each step in the trace includes: the current tape state with positional labels, a sliding-window pass over all three-cell neighborhoods with explicit rule lookup (showing the three cell values, the binary index into the rule table, and the output value), the resulting next tape state, and a formatted print line. Bits are encoded as block characters rather than digits, reserving numerical characters for positional indexing. Rule numbers are converted to their 8-bit binary representation in-context.

We tested all 256 rules (0–255) on random inputs. A tape was scored as correct if and only if its output matched the reference computation from the first token.

The following excerpt shows the beginning of a test run for Rule 90 on a 12-cell tape. The model first decodes the rule number into its 8-bit binary representation and derives the eight neighborhood-to-output mappings, then begins the sliding-window computation:

```
START: MAX 12
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
```

```
TAPE 0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
RULE 90
0/7 90 45 0  -> 0: 4/7 5 2 1  -> 4:
1/7 45 22 1  -> 1: 5/7 2 1 0  -> 5:
2/7 22 11 0  -> 2: 6/7 1 0 1  -> 6:
3/7 11 5 1  -> 3: 7/7 0 0 0  -> 7:
BINARY 7: 6: 5: 4: 3: 2: 1: 0:
RULES FROM BINARY
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
```

After deriving the rule table, the model applies it step by step. Each step shows the three-cell sliding window, the binary index into the rule table, and the resulting output for each cell:

```
TAPE 0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
PRINT 0/12 0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
...
6: 7: 8: 9: 10: 11: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
7: 8: 9: 10: 11: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
8: 9: 10: 11: 3: 4: 5: 6: 7: 8: 9: 10: 11:
9: 10: 11: 4: 5: 6: 7: 8: 9: 10: 11:
10: 11: 5: 6: 7: 8: 9: 10: 11:
TAPE 0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
PRINT 1/12 0: 1: 2: 3: 4: 5: 6: 7: 8: 9: 10: 11:
```

The model continues this process for all 12 steps, producing the full space-time grid. At the end, it reprints the complete grid from memory, reaching back through the full 4,096-token output to copy each line.

2.3 Binary Multiplication

To test whether the model’s computational capacity extends beyond cellular automata to a structurally different class of algorithm, we constructed a tape for binary multiplication. Binary multiplication requires fundamentally different computational primitives than ECA simulation: rather than local neighborhood lookup with synchronous update, it requires iterating over the bits of one operand, computing partial products by shifting and masking, accumulating results with carry propagation, and producing a final output whose bit width is the sum of the operands’ bit widths.

The tape structure follows the same principles as the ECA tape: dense positional indexing, block-character encoding of bits, and structured traces showing each intermediate step. Two input operands (labeled A and B with positional indices) are joined into an output register (labeled O), and the multiplication proceeds bit-by-bit with explicit carry tracking.

We tested on 12 random operand pairs with bit widths up to 27 bits, producing products up to 54 bits.

The following excerpt shows the beginning of a multiplication run. The model first maps the two operands (A, B) into a joint output register (O), then proceeds bit by bit, multiplying each bit of B against all bits of A with explicit carry tracking:

```
START
PREPARE
A0: A1: A2: A3: A4: A5: A6: A7: A8: A9: A10: A11: ...
B0: B1: B2: B3: B4: B5: B6: B7: B8: B9: B10: B11: ...
JOIN
```

A0 00 A1 01 A2 02 ... A26 026
 B0 027 B1 028 B2 029 ... B26 053

Each bit-by-bit step shows the current B bit, the current A bit, the current output register value, and the result of the AND operation with carry propagation:

B26 ■ A26 ■ 053 ■
 053 ■
 B26 ■ A25 ■ 052 ■
 052 ■
 B26 ■ A24 ■ 051 ■
 0 1
 051 ■
 B26 ■ A23 ■ 050 ■
 0 1
 050 ■
 B26 ■ A22 ■ 049 ■
 0 1
 049 ■

The model iterates through all 27 bits of B, and for each, sweeps across all 27 bits of A, accumulating partial products with carries into the 54-bit output register. The largest test case (27×27 bits) required five chained API calls, producing approximately 16,400 output tokens and consuming 159,700 of the model’s 200,000-token context window by the final call. Smaller test cases completed within a single call.

3 RESULTS

3.1 Cellular Automata

Opus produced correct output for all 256 rules tested (256/256, 100%). Three aspects of this result merit emphasis.

Generalization from ten rules to 256. The training examples covered only ten of the 256 possible rules. The model generalized to the remaining 246 rules without error. This is consistent with the model having learned the *algorithm* — three-cell neighborhood lookup, binary rule indexing, synchronous update — rather than memorizing rule-specific input-output patterns. Ten rules exercising the algorithmic primitives in varied combinations were sufficient to constrain the compositional problem; the remaining rules are new parameterizations of an already-learned procedure.

Rule 110. Rule 110, proven Turing complete by Cook (2004), was not among the ten training rules. The model simulated it correctly on a random input. Because Rule 110 can simulate any Turing machine given sufficient tape, and because Opus can simulate Rule 110, the model is empirically capable of universal computation — bounded, as with any physical system, by available memory (here, the context window).

Rule 0. Rule 0 maps every three-cell neighborhood to 0, producing an all-off tape after the first step. This is the cleanest test of algorithmic induction versus distributional approximation. The output of Rule 0 after step one — a uniform blank grid — bears no resemblance to any of the training examples or to typical ECA output patterns. A model relying on distributional similarity to CA-shaped training data would be expected to hallucinate activity.

Test	A bits	B bits	Product	Bit ops	Calls
1	27	27	54	514	5
2	15	14	29	60	1
3	2	1	3	2	1
4	4	4	8	4	1
5	3	3	6	3	1
6	3	2	5	4	1
7	3	2	5	4	1
8	4	3	7	12	1
9	4	3	7	6	1
10	4	3	7	6	1
11	4	3	7	8	1
12	3	2	5	1	1

Opus produced the correct blank output with no errors or hesitation, consistent with execution of the learned update rule rather than interpolation over output distributions.

3.2 Binary Multiplication

Opus produced correct output for all 12 multiplication tests (12/12, 100%), including operands up to 27 bits producing 54-bit products — numbers up to approximately 1.3×10^8 , or roughly 9 decimal digits, with products up to approximately 16 decimal digits. The full test matrix is shown below.

To contextualize this result: at the time of these experiments (May 2024), multi-digit multiplication was widely regarded as a task that exposed fundamental limitations of the transformer architecture. Dziri et al. (2024) reported GPT-4 accuracy of 59% on 3-digit by 3-digit multiplication. Ball et al. (2024) measured GPT-4 at 100% for 2-digit by 2-digit multiplication but only 1.0% for 4-digit by 4-digit. The research frontier was studying 5-digit by 5-digit multiplication on open models, where even with conditioning techniques, accuracy remained low (Stolfo et al., 2024). A specialized tiny transformer trained exclusively for arithmetic achieved 99.9% on 5-digit multiplication — presented as surpassing GPT-4 (Wan et al., 2024). The BigBench Arithmetic benchmark, which tests only up to 5-digit operands, was considered unsolved for the multiplication subtask (Jain et al., 2024).

Our result — 100% accuracy at 9-digit scale on a general-purpose model with no fine-tuning — substantially exceeds this landscape. It is significant for three reasons.

First, binary multiplication is a structurally different algorithm from cellular automata. ECA simulation involves local, synchronous operations: each cell reads its immediate neighbors and applies a lookup. Multiplication requires global coordination: carry propagation connects distant bit positions, partial products must be accumulated across the full width of the output, and the bit-by-bit iteration over one operand creates a sequential dependency structure absent in CAs. The model’s success on both tasks indicates that its in-context learning capacity generalizes across algorithm classes, not just across parameterizations of a single algorithm.

Second, the largest test case (27×27 bits, 514 bit operations) required five chained API calls, producing approximately 16,400 output tokens and consuming 80% of the model’s context window. The chaining produced no accumulated errors: the model resumed computation exactly from the cut point at each continuation, maintaining perfect fidelity across all five calls. This demonstrates that the context window functions as a viable computational tape that can be extended through chaining, and that the model’s execution fidelity is robust to interruption and resumption.

Third, the result reframes what was widely understood as a fundamental architectural limitation. The models tested in prior work were asked to multiply in decimal notation from a conventional prompt, with no intermediate state externalized and no positional anchoring for the attention mechanism. Under those conditions, the task is indeed intractable beyond a few digits. But the same underlying model, given a structured tape with binary encoding, dense positional indexing, and explicit intermediate steps, achieves perfect accuracy on operands roughly twice the digit count of anything previously tested. The limitation is not in the transformer’s computational capacity; it is in how the problem is presented.

4 DISCUSSION

4.1 Bridging Theory and Empirical Behavior

The results reported here occupy a specific position relative to prior theoretical work. Pérez et al. (2021) proved that a transformer *can be constructed* to simulate a Turing machine. Our results show that a production transformer, trained by gradient descent on natural language without any modifications for computational tasks, *already can* simulate a Turing machine when given appropriate in-context examples. The universality is not engineered; it is elicited from a model whose computational circuits emerged from the selection pressure of next-token prediction over a broad distribution.

This distinction matters because theoretical Turing completeness constructions for neural networks have existed for decades without bearing on what trained models actually do. The contribution here is empirical: demonstrating that the gap between theoretical capacity and practical behavior can be closed by in-context learning alone, without architectural modifications, fine-tuning, or chain-of-thought scaffolding.

4.2 In-Context Learning as Algorithmic Induction

The model’s performance across both experiments constitutes exact algorithmic induction. In the ECA case, twelve examples covering ten rules are sufficient for the model to learn the general update algorithm and apply it to all 256 rules. In the multiplication case, the model learns the binary multiplication algorithm from structured examples and applies it to random operands it has never seen.

In both cases, the model did not memorize input-output pairs; it learned a *procedure* and applied it to novel in-

puts. The mechanism by which this induction occurs is the in-context learning capacity of the transformer, operating over circuits — specific configurations of attention heads and MLP layers — that implement the requisite atomic operations (positional lookup, binary switching, conditional value copying, carry propagation). These circuits, which have been characterized by recent mechanistic interpretability work (Anthropic, 2025), provide the computational primitives; the in-context examples compose them into the specific algorithm.

4.3 Arithmetic and the Compositional Limitation Claim

The multiplication result has implications beyond arithmetic per se. Dziri et al. (2024) used multi-digit multiplication as a central exhibit in arguing that transformers face fundamental limits on compositional reasoning — that performance “rapidly decays with increased task complexity” because the model must reduce multi-step compositional reasoning to linearized subgraph matching. This framing was influential: it suggested that the errors observed on multiplication reflected something intrinsic to the transformer architecture, not merely a prompt engineering problem.

Our results suggest a different interpretation. The computational circuits required for multiplication — binary switching, positional lookup, conditional value copying, carry propagation — are present in the trained model. What fails under conventional prompting is not the computation itself but the *interface* between the task representation and the attention mechanism. Decimal digit tokenization creates ambiguous positional relationships. The absence of externalized intermediate state forces the model to track carries and partial products entirely within a single forward pass. And without positional anchors, attention has nothing to lock onto as sequence length grows.

When these interface problems are resolved by tape design — binary encoding eliminates tokenization ambiguity, structured traces externalize intermediate state, dense positional indexing gives attention explicit targets — the model multiplies numbers far larger than anything reported in the literature, with perfect accuracy. The compositional limitation identified by Dziri et al. is real, but it characterizes the interaction between the task format and the attention mechanism, not the computational capacity of the architecture.

4.4 Attention Saturation and Tape Design

The primary practical constraint on computation is not the model’s ability to learn algorithms but the fidelity of execution over long sequences. As Merrill (2023) has characterized, attention in trained transformers can become unsaturated as sequence length grows: rather than attending sharply to the indices that maximize the attention score, attention distributes across neighboring positions, causing the model to misread its own previously emitted values.

This manifests as copy errors: the model applies the correct logic to incorrectly read data, producing a program

Call	Input	Output	Total
Initial	30.0K	4.0K	34.0K
Continue 1	33.8K	4.1K	37.9K
Continue 2	71.7K	8.2K	79.9K
Continue 3	113.6K	12.3K	125.9K
Continue 4	159.7K	16.4K	176.1K

trace that diverges from the true computation. The heavily indexed tape structure used in both experiments is a prompt-level countermeasure against this architectural limitation. Dense positional labels give attention heads stronger targets, reducing leakage onto adjacent indices. The tape engineering is, in effect, a manual intervention on circuit-level attention dynamics.

Merrill’s proposed fix — scaling attention scores by $\log n$, where n is the context length — would maintain saturation as sequences grow, but requires training with the modification rather than applying it at inference. For pre-trained models, tape design remains the primary tool for managing execution fidelity.

4.5 Computational Capacity

This work establishes that a production transformer can function as a universal computer and can learn algorithms across different computational classes. It does not establish the practical limits of that capacity. The algorithms tested here — ECA simulation and binary multiplication — are computationally simple relative to what the architecture may be capable of learning.

The response chaining technique extends the effective tape well beyond a single API call’s output budget. The token budget for the largest multiplication test (27×27 bits) illustrates the scaling:

By the final call, the computation occupies 176,000 of the model’s 200,000-token context window. The model uses approximately 30,000 tokens for training examples, leaving approximately 170,000 tokens of effective tape when chaining is used — roughly 40 times the single-call output budget. Whether the model can learn substantially more complex programs within this budget, and at what point the attention saturation problem becomes prohibitive at these sequence lengths, remains an open empirical question.

5 RELATED WORK

Theoretical Turing completeness. Pérez et al. (2021) proved transformer Turing completeness with positional encodings and unbounded precision. Li and Wang (2025) extended this to constant bit-size transformers with chain-of-thought. Merrill (2023) characterized saturated transformers as constant-depth threshold circuits.

LLM arithmetic and compositionality. Dziri et al. (2024) argued that transformer performance on compositional tasks including multiplication rapidly decays with task complexity, attributing this to fundamental architec-

tural limitations. Ball et al. (2024) measured GPT-4 at 1% accuracy on 4-digit by 4-digit multiplication. Wan et al. (2024) achieved 99.9% on 5-digit multiplication with a specialized transformer trained exclusively for arithmetic. Stolfo et al. (2024) studied the difficulty structure of multi-digit multiplication for language models. These results collectively established multiplication as a canonical example of transformer limitations — a framing our results complicate.

Mechanistic interpretability. Anthropic’s circuit tracing work (2025) has identified interpretable computational circuits in production language models, providing a mechanistic basis for understanding how in-context algorithmic behavior is implemented at the level of attention heads and MLP layers.

Elementary cellular automata. Wolfram (2002) systematically classified all 256 elementary rules. Cook (2004) proved Rule 110 Turing complete. The connection between cellular automata and computational universality provides the theoretical foundation for using ECA simulation as a test of universal computation.

6 CONCLUSION

We have shown that Claude 3 Opus, a production large language model, can simulate all 256 elementary cellular automata rules — including the Turing-complete Rule 110 — and execute binary multiplication on operands up to 27 bits, by learning each algorithm from in-context examples alone. No architectural modifications, fine-tuning, or chain-of-thought reasoning are required. The model induces each algorithm from patterned examples and executes it via autoregressive generation with perfect accuracy.

This constitutes, to our knowledge, the first empirical demonstration of universal computation in a production transformer model. The results bridge theoretical Turing completeness proofs for the transformer architecture with the observed capabilities of trained systems, and suggest that the computational circuits necessary for general-purpose computation emerge naturally from gradient descent on sufficiently broad training distributions.

REFERENCES

- [1] J. Pérez, P. Barceló, and J. Marinkovic, “Attention is Turing complete,” *Journal of Machine Learning Research*, vol. 22, no. 75, pp. 1–35, 2021.
- [2] M. Cook, “Universality in elementary cellular automata,” *Complex Systems*, vol. 15, no. 1, pp. 1–40, 2004.
- [3] Q. Li and Y. Wang, “Constant bit-size transformers are Turing complete,” *arXiv:2506.12027*, 2025.
- [4] W. Merrill, “Saturated transformers are constant-depth threshold circuits,” *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1303–1317, 2023.

- [5] N. Dziri, X. Lu, M. Sclar, X. L. Li, L. Jiang, B. Y. Lin, S. Welleck, P. West, C. Bhagavatula, R. Le Bras, J. D. Hwang, S. Sanyal, X. Ren, A. Ettinger, Z. Harchaoui, and Y. Choi, “Faith and fate: Limits of transformers on compositionality,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [6] T. Ball, S. Chen, and C. Herley, “Can we count on LLMs? The fixed-effect fallacy and claims of GPT-4 capabilities,” *arXiv:2409.07638*, 2024.
- [7] Z. Wan, Y. Liu, J. He, and Z. Lin, “Dissecting multiplication in transformers: Insights into LLMs,” *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [8] A. Stolfo, Y. Belinkov, and M. Sachan, “Language models do hard arithmetic tasks easily and hardly do easy arithmetic tasks,” *arXiv:2406.02356*, 2024.
- [9] N. Jain, R. Schwarzschild, A. Addepalli, and T. Goldstein, “IGC: Integrating a gated calculator into an LLM to solve arithmetic tasks reliably and efficiently,” *arXiv:2501.00684*, 2024.
- [10] Anthropic, “Circuit tracing: Revealing computational graphs in language models,” *transformer-circuits.pub*, 2025.
- [11] S. Wolfram, *A New Kind of Science*. Wolfram Media, 2002.

DATA AVAILABILITY

All code, training tapes, test harnesses, and complete unedited run logs are publicly available at <https://github.com/EvolvingPrograms/turing>. The repository contains the tape construction code, the training tapes used as in-context examples for both experiments, and the automated test harness. Complete timestamped CI output from the April 22, 2024 cellular automata test run (256/256) is available at <https://gist.github.com/ctjlewis/5f72181923fea4f933348b561401a411/raw/80758fb9813c8464eb62481bc51c0b0ec2a58568/automata.txt> and the May 14, 2024 multiplication test run (12/12) at <https://gist.github.com/ctjlewis/5f72181923fea4f933348b561401a411/raw/80758fb9813c8464eb62481bc51c0b0ec2a58568/multiplication.txt>. All test runs were executed via GitHub Actions on `claude-3-opus-20240229` at temperature 0.